OACAL: Finding Module-Consistent Solutions to Weaken User Obligations

JIANG Pengcheng (1W17BG06)

Honiden-Tei Lab.

rexjiang@fuji.waseda.jp

OUTLINE

Background

Problems

Proposal

Evaluation

Conclusion

BACKGROUND

Background

- Original paper of OASIS [1]

2020 IEEE 28th International Requirements Engineering Conference (RE)

OASIS: Weakening User Obligations for Security-critical Systems

Thein Than Tun* Amel Bennaceur* Bashar Nuseibeh†
*The Open University, UK *†Lero, Ireland
firstname.lastname@open.ac.uk

[1] T. T. Tun, A. Bennaceur and B. Nuseibeh, "OASIS: Weakening User Obligations for Security-critical Systems," 2020 IEEE 28th International Requirements Engineering Conference (RE), 2020, pp. 113-124.

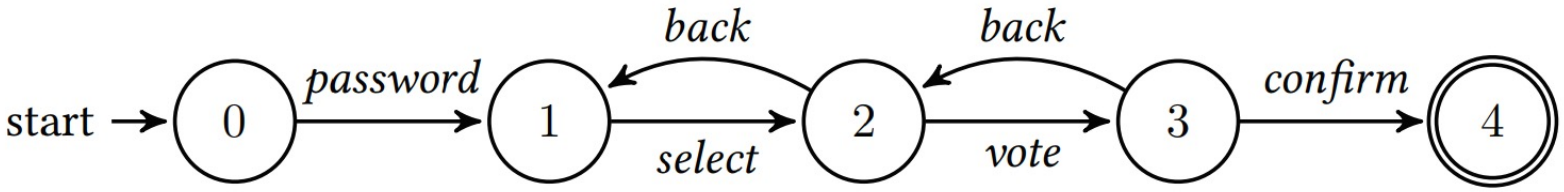
Background

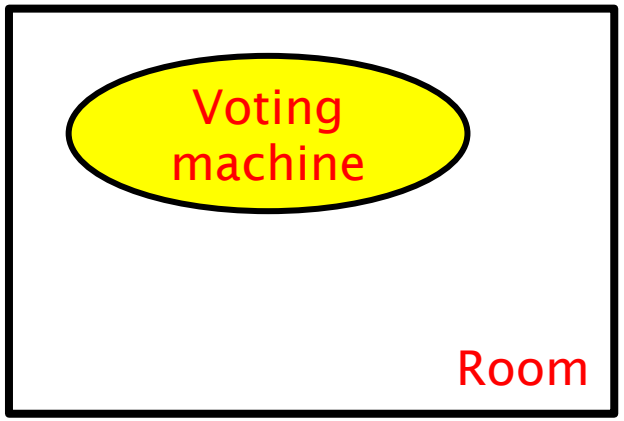
- User Obligations (Definition)

A system has assumption on user's behavior.

For example, a system of a e-voting machine:

Behavior model:





A voting machine in a room

Machine's assumption:
(1) User would follow the designated behavior model without any exception.
(2) The user would be the only operator of it.

User Obligation:
Staying in the room until completing the whole voting procedure by performing "confirm".

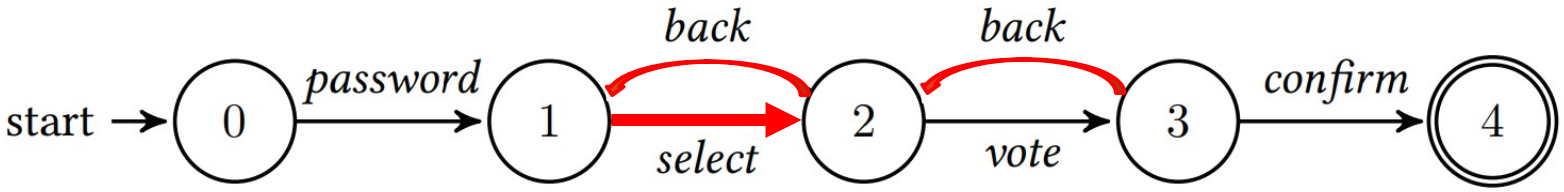
Background

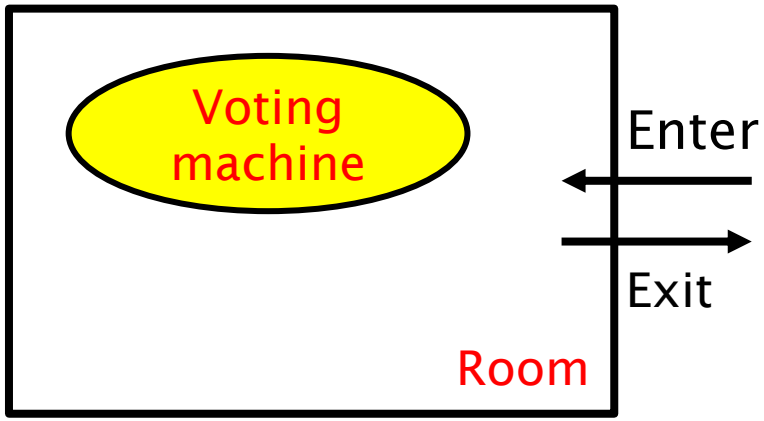
- User Obligations (Weakened Condition)

A system has assumption on user's behavior.

For example, a system of a e-voting machine:

Behavior model:



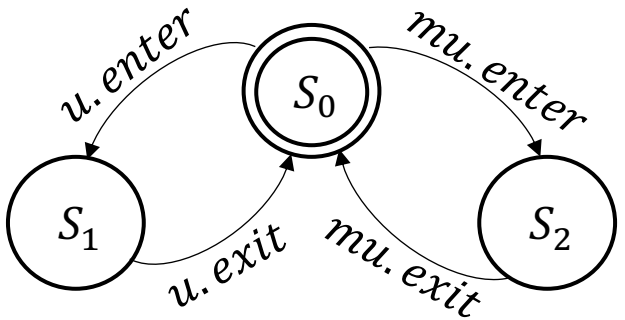


A voting machine in a room

What if the user perform some actions out of the assumption by machine?

Like "Enter" and "Exit"?

➡ "Vote flipping" may happen!

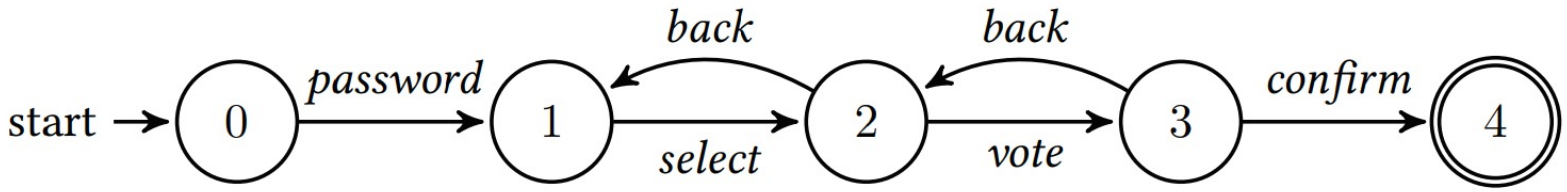


u: user
mu: malicious user

Background

- User Obligations (Weakened Condition)

  Behavior model:



What if the user perform some actions out of the assumption by machine?

= Weakening of UO

↓ cause

"Vote flipping"

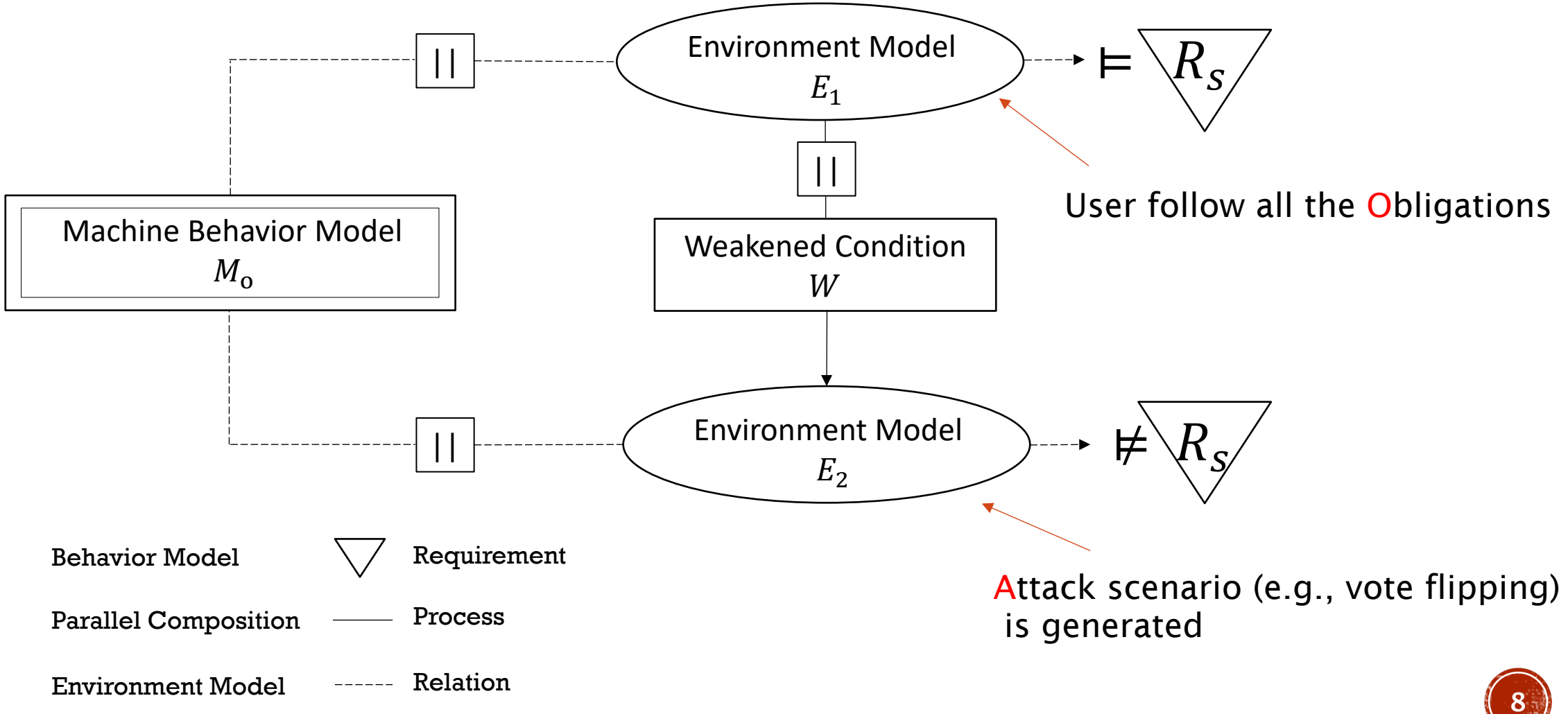
= Violation to the requirement

There is a need to redesign the original system! → OASIS

Background

- OASIS (Obligations, Attack scenario, SpecIfication abstraction, and Synthesis)

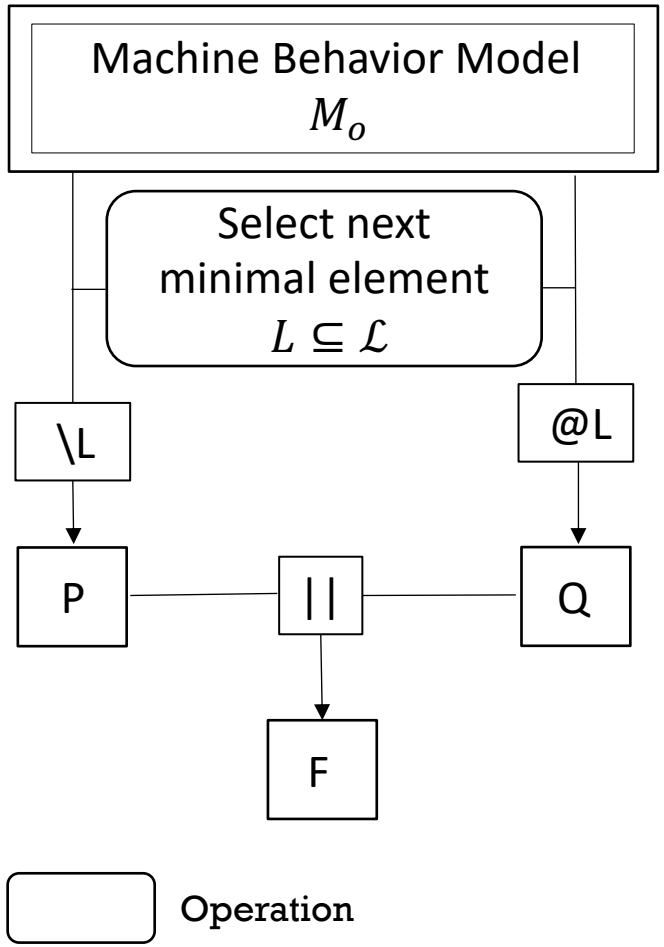
Obligations, Attack scenario:



Background

- OASIS (Obligations, Attack scenario, Specification abstraction, and Synthesis)

Specification abstraction:

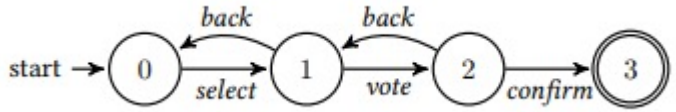


"Recomposition of the specification"

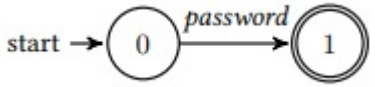
1. Select a minimal subset L of the Language set $\mathcal{L}$ of M_o
   For example, if we set $L = password$.

2. Obtain two abstractions P and Q with the selected L

P $\quad P = M_o \setminus \{L\}$



Q $\quad Q = M_o @ \{L\}$



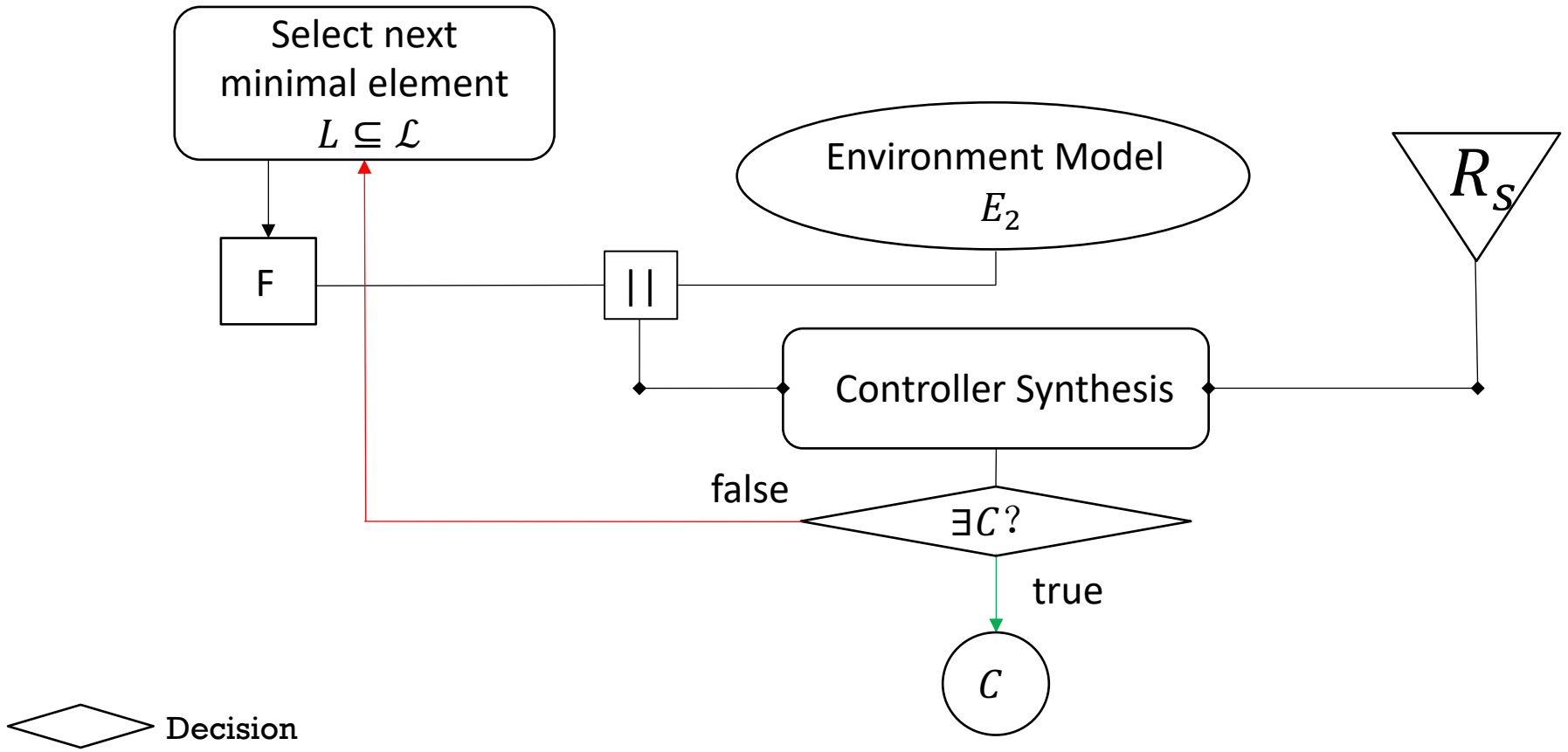
3. Compose them together to include all possible sequences

$F \leftarrow P \;||\; Q$

Background

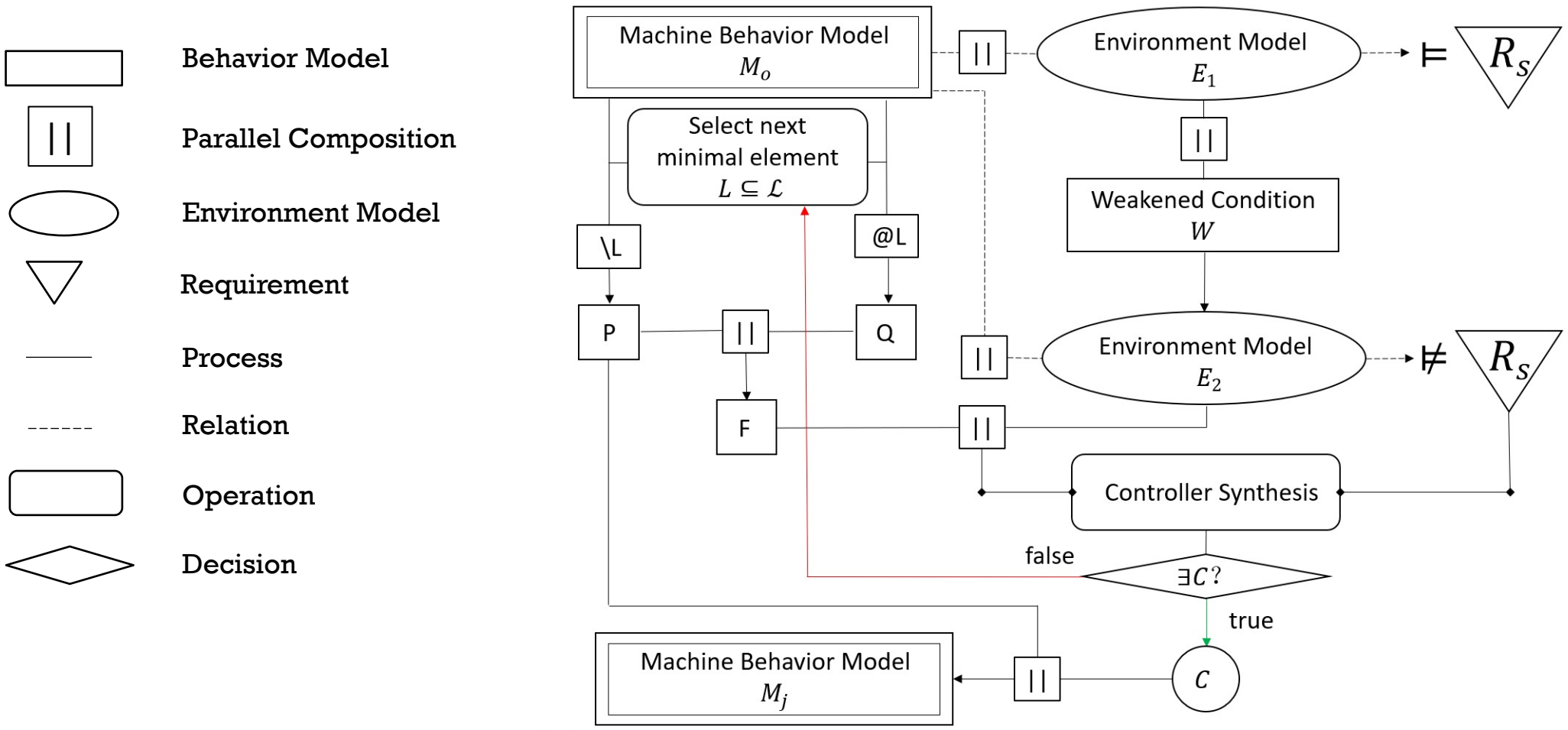
- OASIS (Obligations, Attack scenario, SpecIfication abstraction, and Synthesis)

Synthesis:



Background

- OASIS (Obligations, Attack scenario, SpecIfication abstraction, and Synthesis)

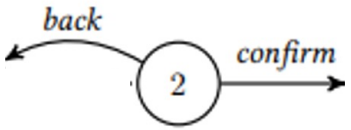


PROBLEMS

Problems

1. Careless of Module Consistency

Example of a module:

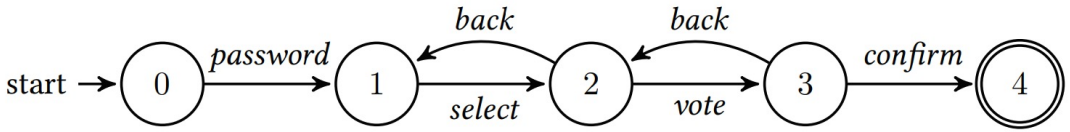


All modules in M_o:
$\{password\}, \{select\}, \{vote, back\}, \{confirm, back\}$

Module-consistent: all modules maintain the same after revision.

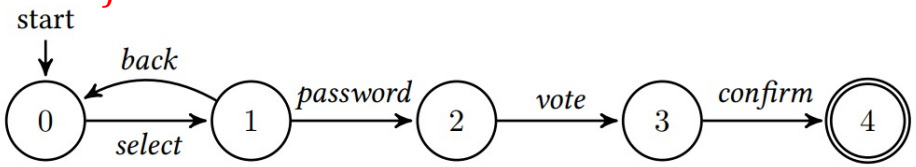
→ In this case, module consistency only remains in revision (c)

Original Behavior Model M_o

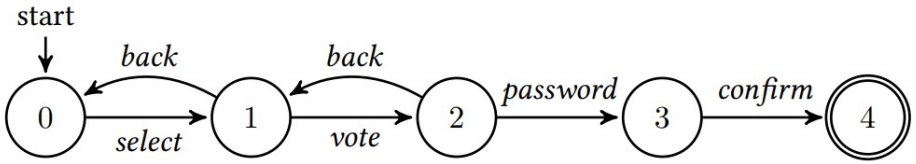


Revisions M_j

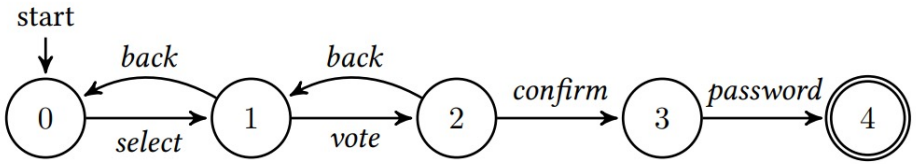
(a)



(b)



(c)



Problems

2. High Time Consumption

Table . Distribution of non-redundant revisions by OASIS

m_{m_L} \ $\mathcal{A}_{m\_L}$	1-action	2-actions	3-actions	4-actions	Sum	Index of the last unrepeated revision	Index of the middle unrepeated revision
4	10/16	4/18	-	-	14/34	28	12
5	17/25	25/100	-	-	42/125	118	35
6	26/36	79/225	27/200	-	132/461	421	139
7	37/49	188/441	204/1225	-	429/1715	1693	471
8	50/64	380/784	766/3136	234/2450	1430/6434	6224	1631
9	65/81	689/1296	2158/7056	1950/15876	4862/24309	24236	7171

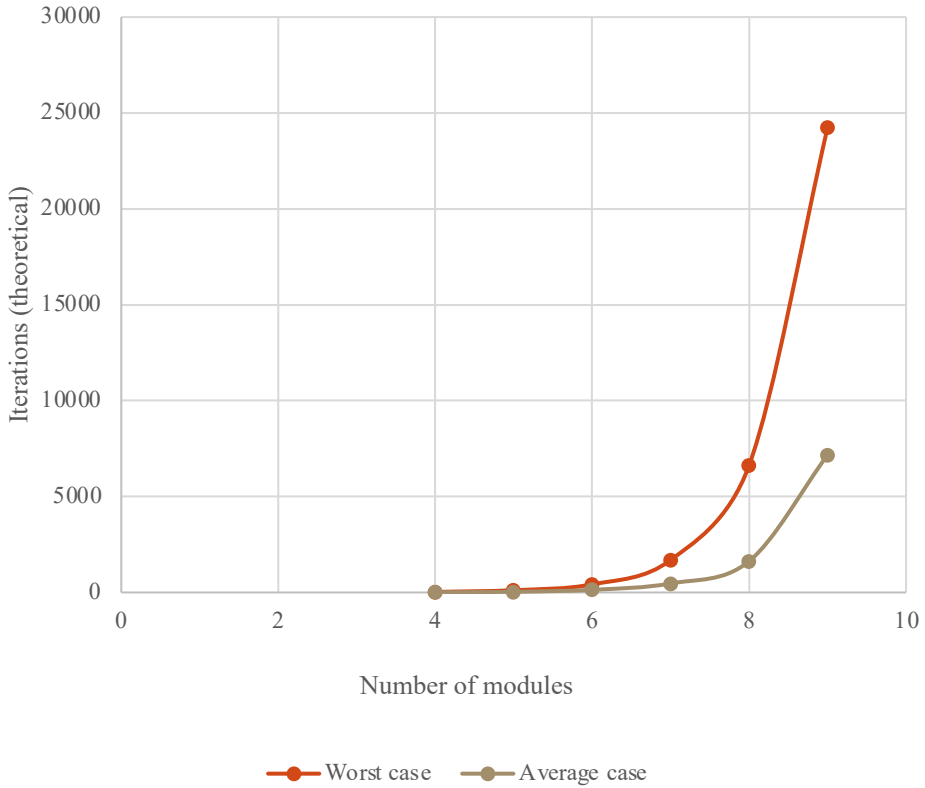


Fig. Time consumption for OASIS to find module-consistent solution vs. module numbers

The iterations needed to get a module-consistent specification that can satisfy the requirements are exponentially increased with module numbers from 4 to 9 in both worst case and average case.

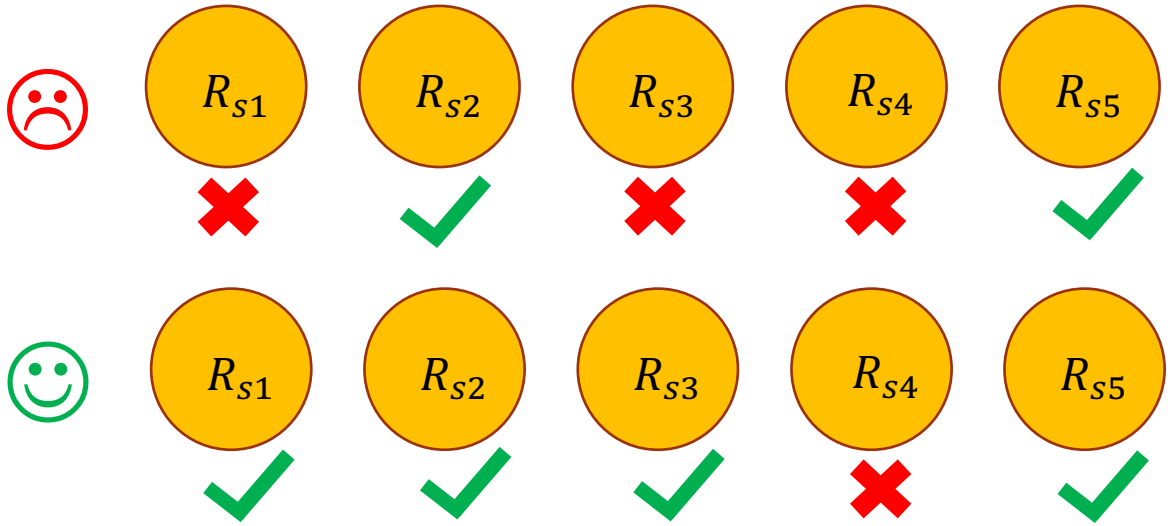
Problems

3. No requirement degradation

OASIS approach only returns the optimal M_j that can surely pass all the requirement checking. Otherwise, return Fail.

Admittedly, it is good when there is only one requirement. However, what if plenty of requirements are checked at the same time? While some of them are weighted differently?

```
7   | synthesise C such that C, F, E2 ⊨ Rs;
8   | if C ≠ Null then
9   |   M2 ← C || M;
10  |   return M2;
11  | end
12  end
13  return Fail;
```



(What if the designer accept this?)

We want "Hope for the best, Prepare for the worst" [3]

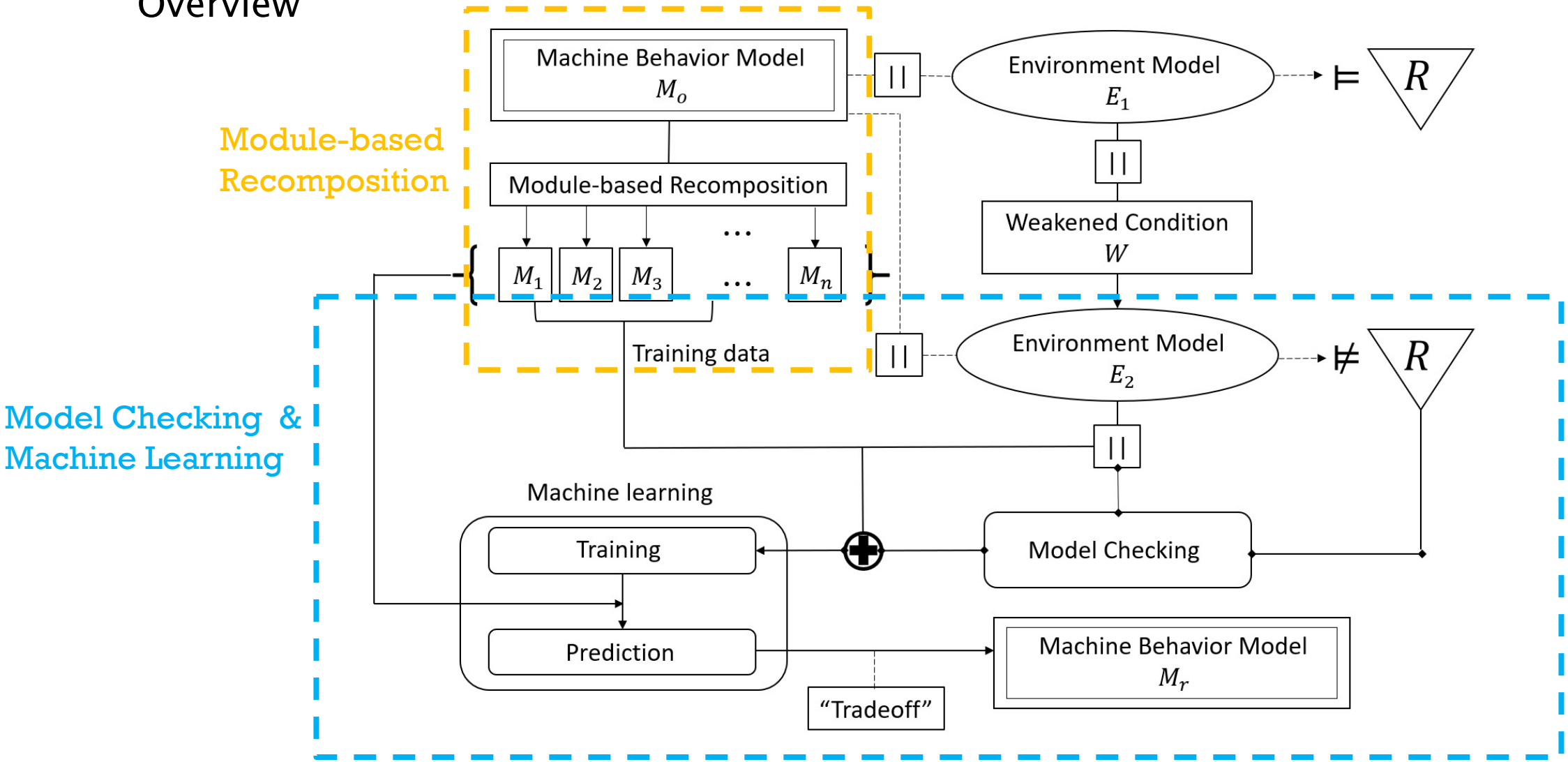
[3] N. D'Ippolito, V. Braberman, J. Kramer, J. Magee, D. Sykes, and S. Uchitel, "Hope for the best, prepare for the worst: Multi-tier control for adaptive systems," in *Proceedings of the 36th International Conference on Software Engineering*, ICSE 2014, (New York, NY, USA), pp. 688–699, ACM, 2014.

PROPOSAL

Proposal

- OACAL(Obligations, Attack scenario, model Checking, And machine Learning)

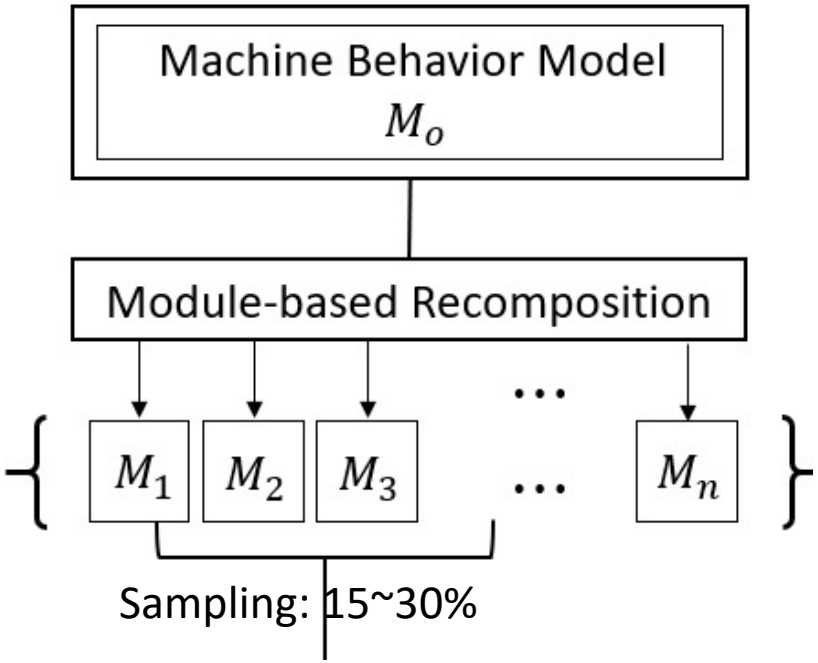
Overview

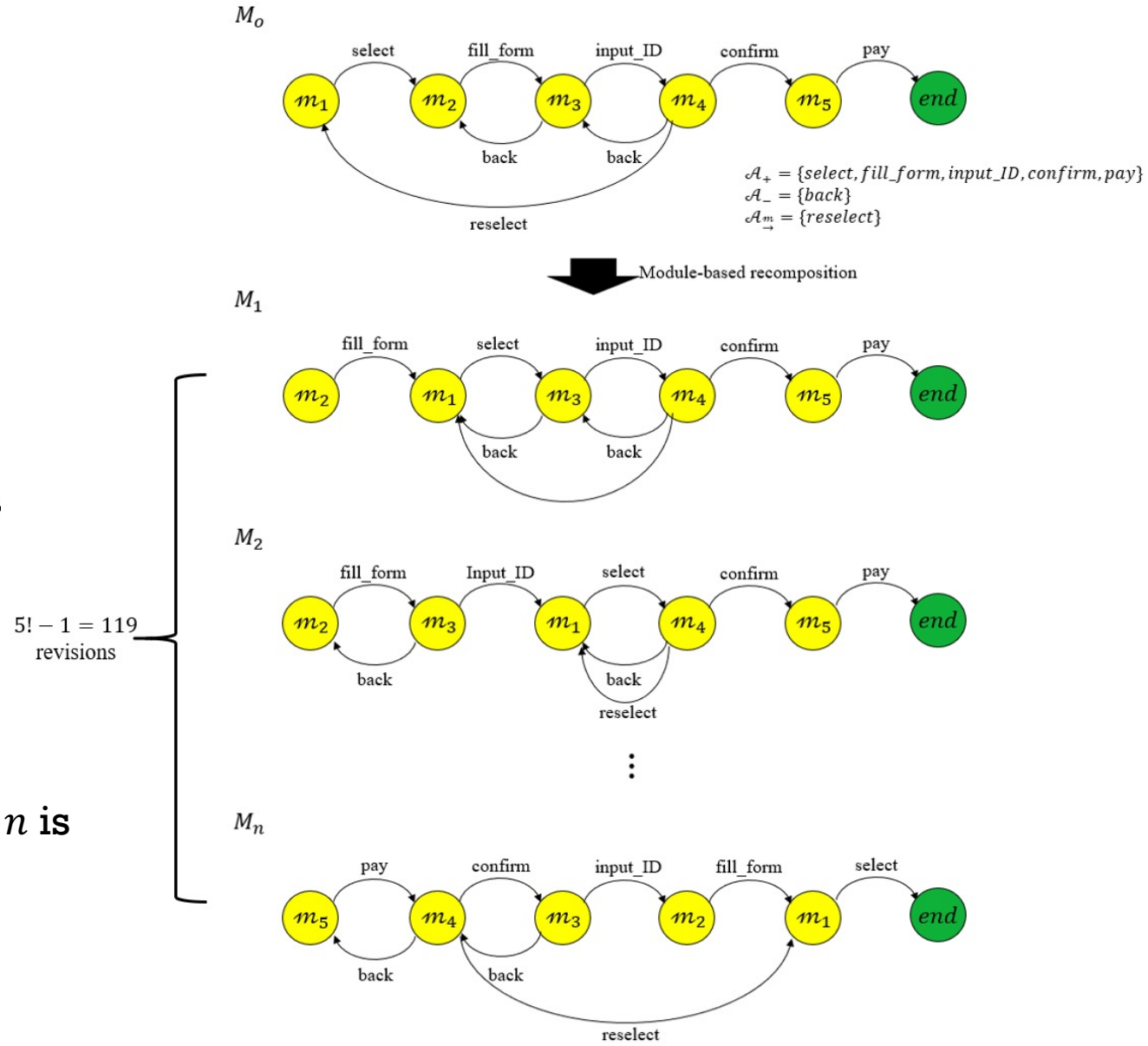


Proposal

- OACAL(Obligations, Attack scenario, model Checking, And machine Learning)

Module-based Recomposition





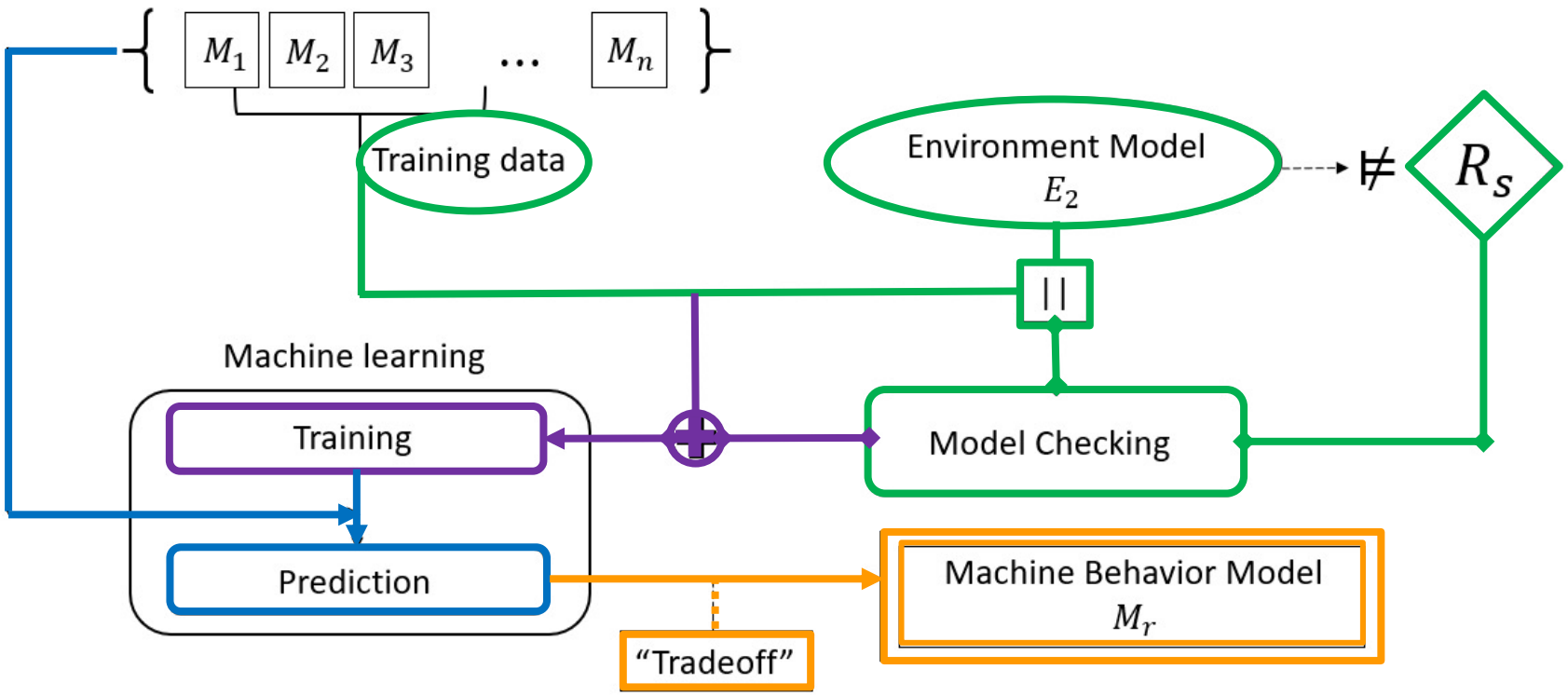
There would be $(n! - 1)$ revisions where n is the number of modules.

In this example: $5! - 1 = 119$

Proposal

- OACAL(Obligations, Attack scenario, model Checking, And machine Learning)

  Model Checking and Machine Learning



1. Do model checking for the randomly selected specifications

2. Append the model checking results as labels to the specifications, and start training the model

3. Do predictions to all the specifications with the trained model

4. Degrade (if needed) the requirements and pick the optimal specification M_r

EVALUATION

Evaluation

Research Questions

RQ1.
How well can OACAL generate more module-consistent revisions than OASIS?

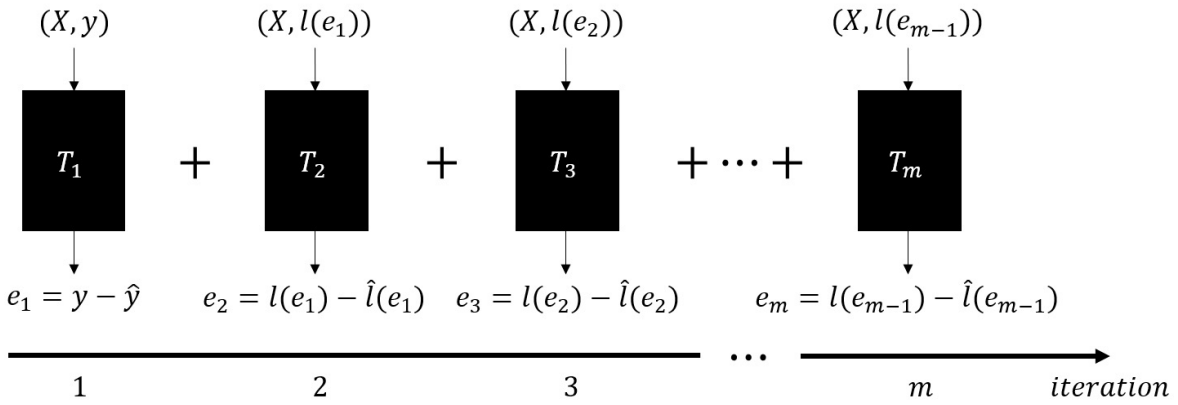
RQ2.
How much is OACAL faster than OASIS on finding solutions?

RQ3.
How requirement tradeoff benefits on finding better revisions?

Evaluation

(1) Validity of the approach.

Based on the testing results, we use Gradient Boosted Trees as the machine learning algorithm:



Gradient Boosted Trees

Accuracy (with training data rate = 0.3):

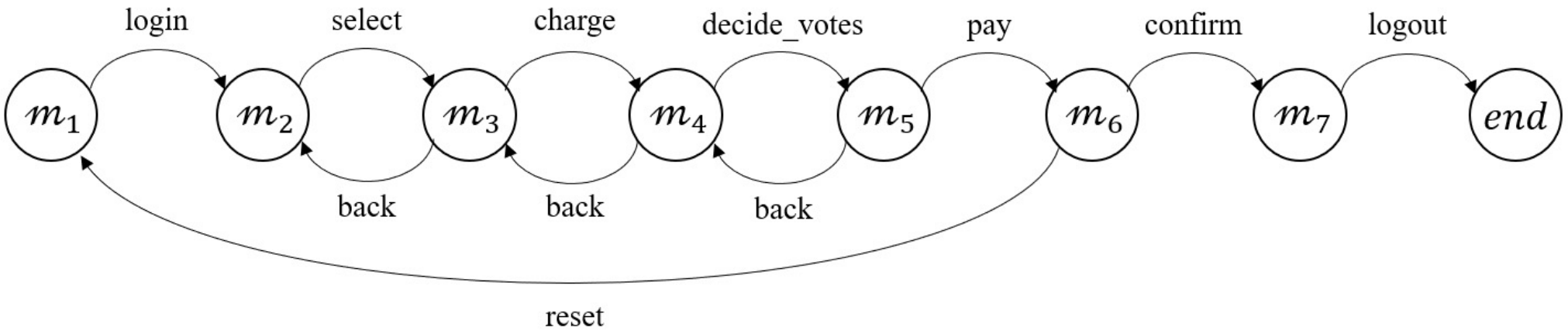
```
In [50]: model_boosted_1.evaluate(test_data)
Out[50]: {'accuracy': 1.0,
          'auc': 1.0000000000000007,
In [51]: model_boosted_2.evaluate(test_data)
Out[51]: {'accuracy': 0.9988509049123815,
          'auc': 1.0000000000000004,
In [52]: model_boosted_3.evaluate(test_data)
Out[52]: {'accuracy': 0.9887963228957196,
          'auc': 0.9994550648913532,
```

```
In [53]: model_boosted_4.evaluate(test_data)
Out[53]: {'accuracy': 0.9919563343866705,
          'auc': 0.998818099238099,
In [54]: model_boosted_5.evaluate(test_data)
Out[54]: {'accuracy': 0.9428325193909796,
          'auc': 0.9875897960253787,
In [55]: model_boosted_6.evaluate(test_data)
Out[55]: {'accuracy': 0.9913817868428613,
          'auc': 0.9993736202268535,
```

Evaluation

(2) Evaluation with a motivation example

Motivation example: Pay & Vote System



Security Requirements:

1. Vote Flipping
2. Personal Information Leaked
3. Number of Votes Changed

Functional Requirements:

1. "ChargeAfterLogin"
2. "NoConfirmAfterSelectUntilVote"
3. "PayAfterCharge"

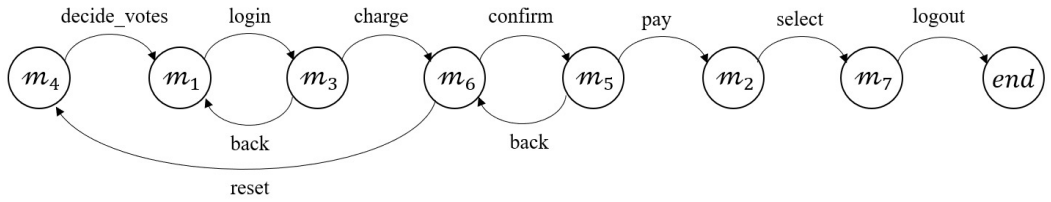
Evaluation

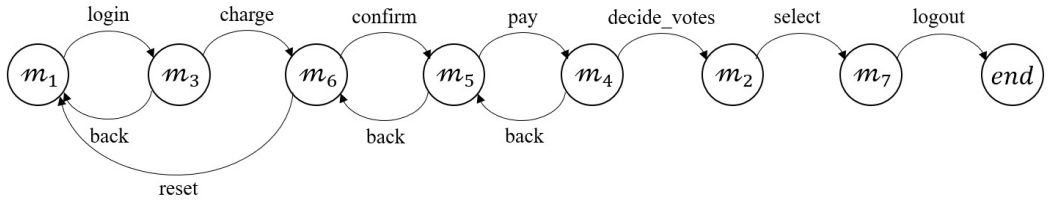
(2) Evaluation with a motivation example

Result by OASIS
- No module-consistent solution could be given

RQ1. - How well can OACAL generate **more revisions** (module-consistent) than OASIS?

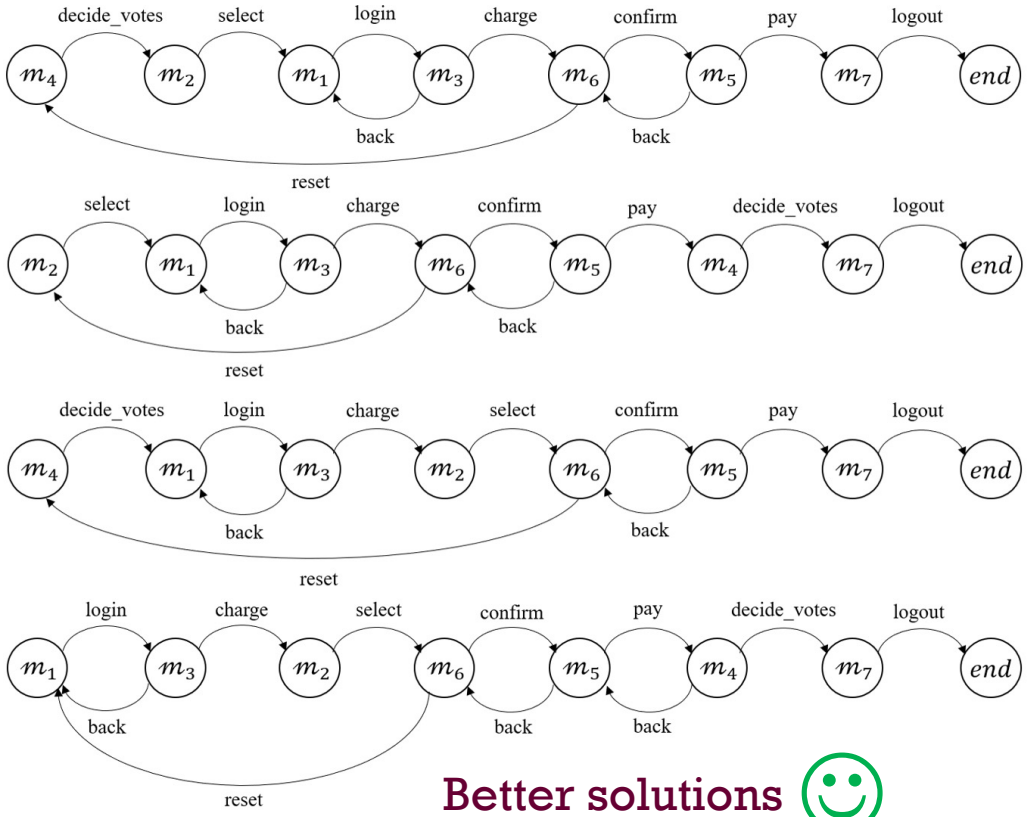
Result by OACAL
- Before requirement degradation:





No desirable solutions 😐

- After requirement degradation:



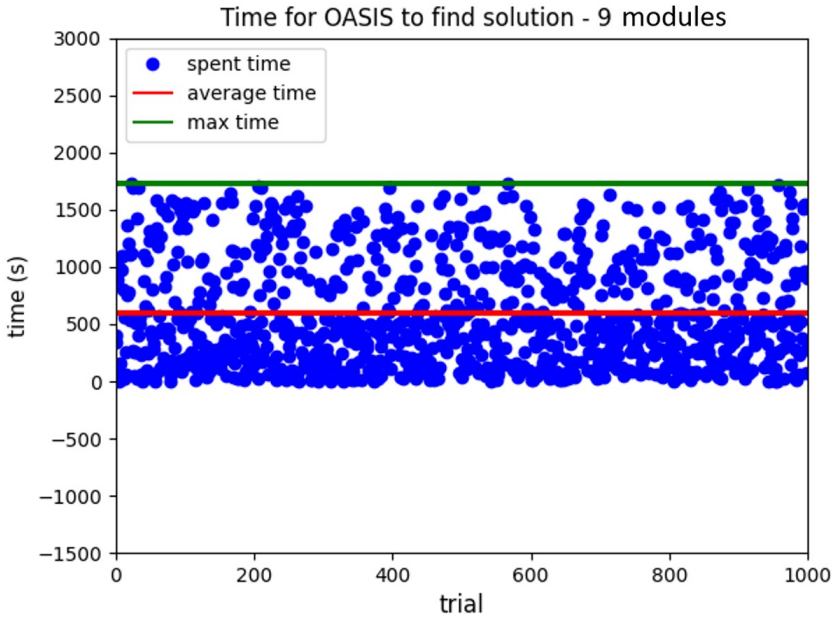
Better solutions 😊

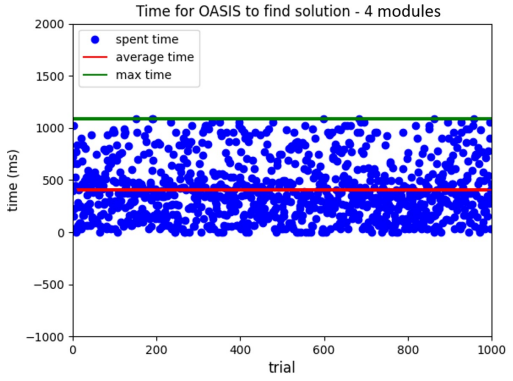
RQ3 - How **degradation** effectively affects on **finding better revisions**?

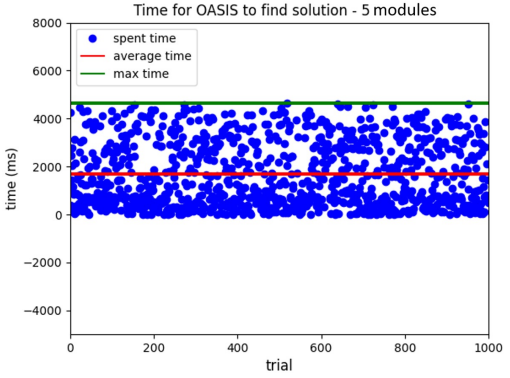
Evaluation

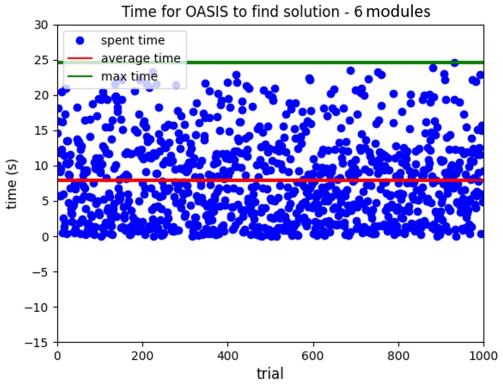
(3) General evaluation on performance

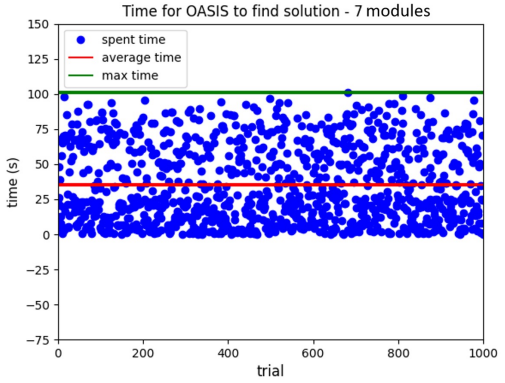
- Searching solution by OASIS

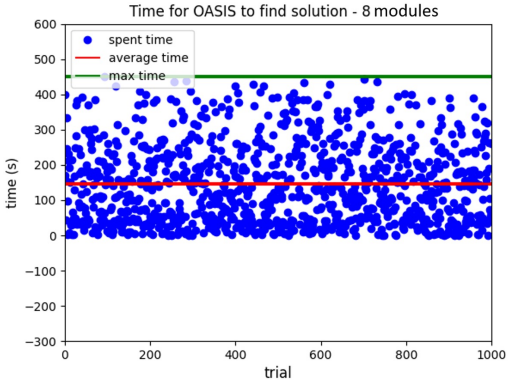


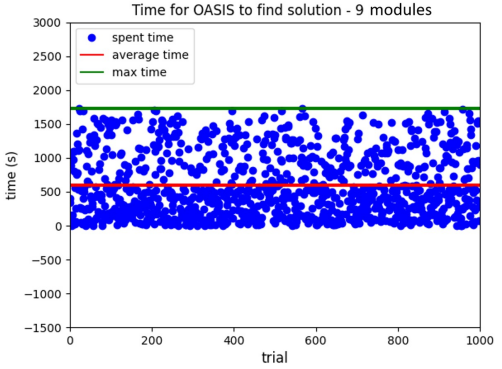








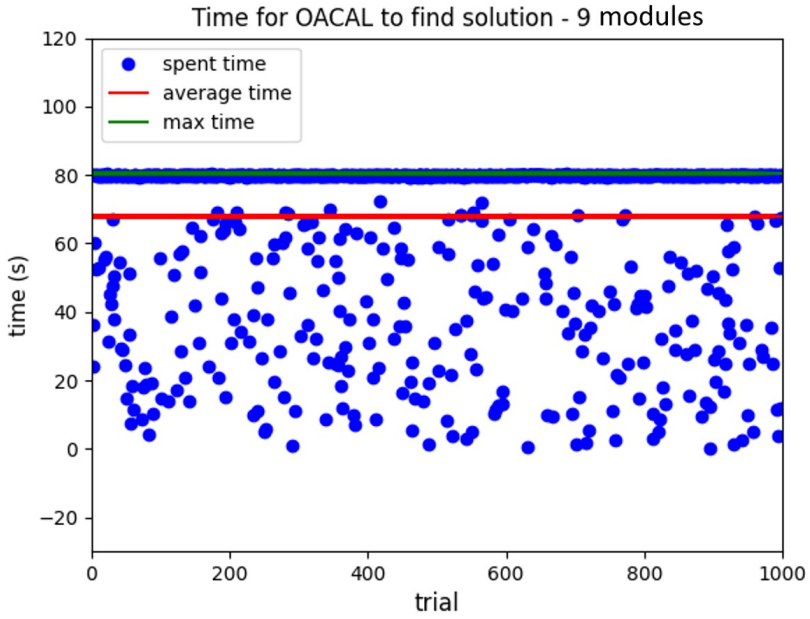


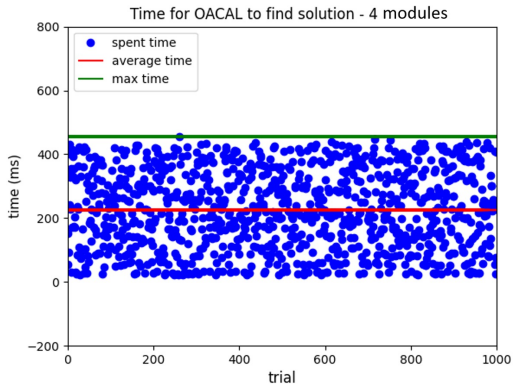


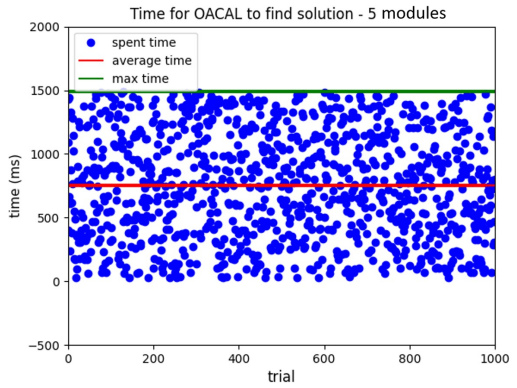
Evaluation

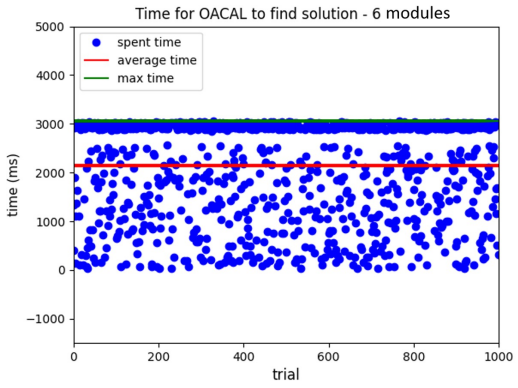
(3) General evaluation on performance

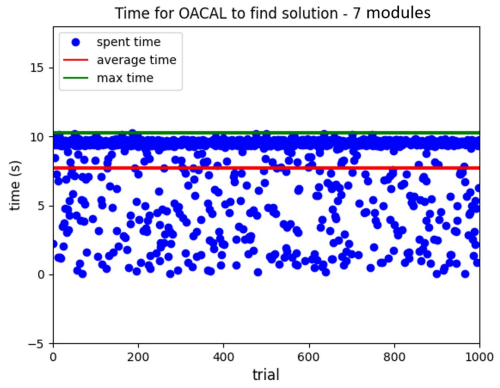
- Searching solution by OACAL

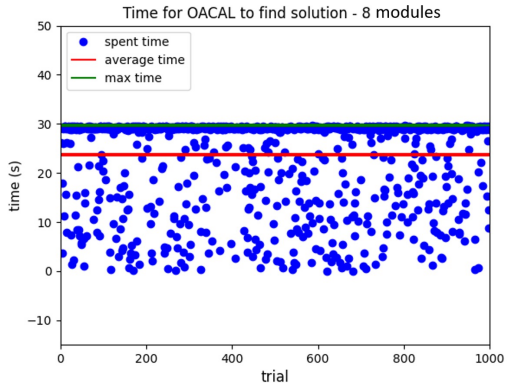


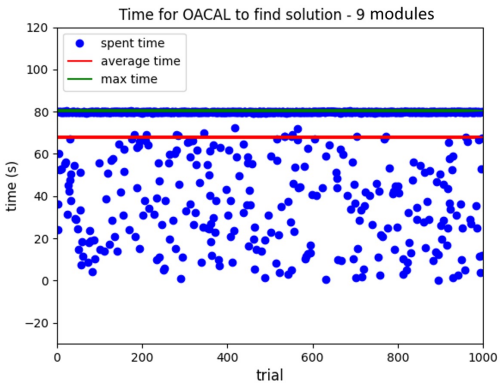










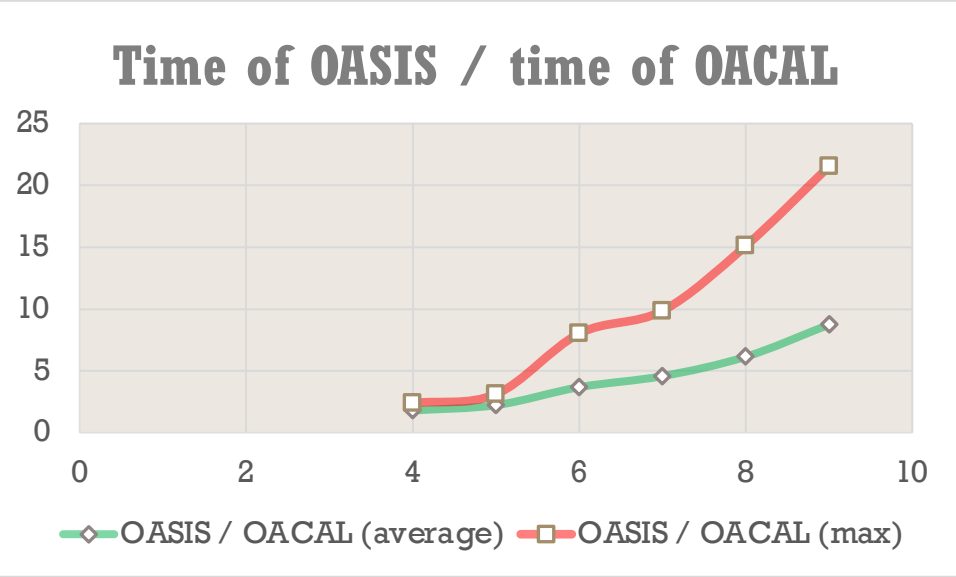


Evaluation

(3) General evaluation on performance

- Comparison

Module numbers	OASIS average time (ms)	OACAL average time (ms)	OASIS max time (ms)	OACAL max time (ms)
4	404.132	224.658	1089	454
5	1672.63	749.972	4636	1488
6	7887.748	2140.88	24574	3063
7	35250.551	7702.507	101008	10243
8	146304.617	23807.490	450102	29793
9	592524.231	67803.626	1731249	80387



RQ2. - How much is OACAL **quicker** than OASIS on **finding solutions**?

OACAL can find module-consistent revision much faster than OASIS for two reasons:

1. No redundant revision
2. Utilizing machine learning to speed up

CONCLUSION

Conclusion

- **RQ1.**
- How well can OACAL generate **more revisions** (module-consistent) than OASIS?
  - In some cases, OASIS cannot generate even one module-consistent revision, while OACAL can generate several ones.
  - OACAL's coverage of revisions is much larger than that of OASIS.

- **RQ2.**
- How much is OACAL **quicker** than OASIS on **finding solutions**?
  - We have demonstrated that, when the coverage is same, OACAL is much faster than OASIS on finding module-consistent solution, and it would be more obvious with the increment of the number of modules.

- **RQ3.**
- How **degradation** effectively affects on **finding better revisions**?
  - In some cases, a solution that can satisfy all the requirements is not always better. Degradation of the requirements may help us find an optimal one.

References

[1] T. T. Tun, A. Bennaceur and B. Nuseibeh, "OASIS: Weakening User Obligations for Security-critical Systems," 2020 IEEE 28th International Requirements Engineering Conference (RE), 2020, pp. 113-124.

[2] 28th IEEE International Requirements Engineering Conference, Available: https://re20.org/.

[3] N. D'Ippolito, V. Braberman, J. Kramer, J. Magee, D. Sykes, and S. Uchitel, "Hope for the best, prepare for the worst: Multi-tier control for adaptive systems," in Proceedings of the 36th International Conference on Software Engineering, ICSE 2014, (New York, NY, USA), pp. 688–699, ACM, 2014.

[4] Rosaria Silipo, "*Ensemble Models: Bagging & Boosting – The Theory and the Practice with KNIME Analytics Platform*", Analytics Vidhya, Mar 19, 2020. Available: https://medium.com/analytics-vidhya/ ensemble-models-bagging-boosting-c33706db0b0b

[5] XGBoost developers, *xgboost, Release 1.5.0-dev*, Jun 11, 2021.

Appendix

Is larger coverage always better?

The answer is “No”, and we do have some cases that larger coverage could not help, listed as follows:

I. There is no solution in all revisions generated by the module-based recomposition for the given conditions (original behavior model and requirements).
II. The designer only wants the solution in OASIS’s coverage so that any revisions outside the scope would not be considered.
III. The module consistency does not matter to the designer so that he/she may find solution by replacing the actions in a module to another or even removing them.

Appendix

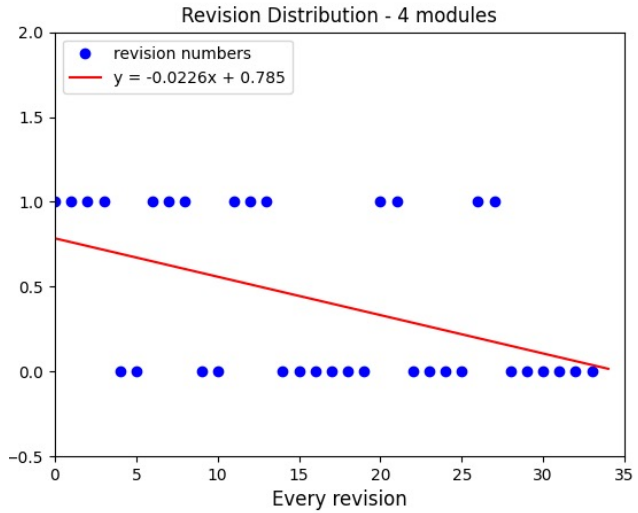
Validity of requirement degradation

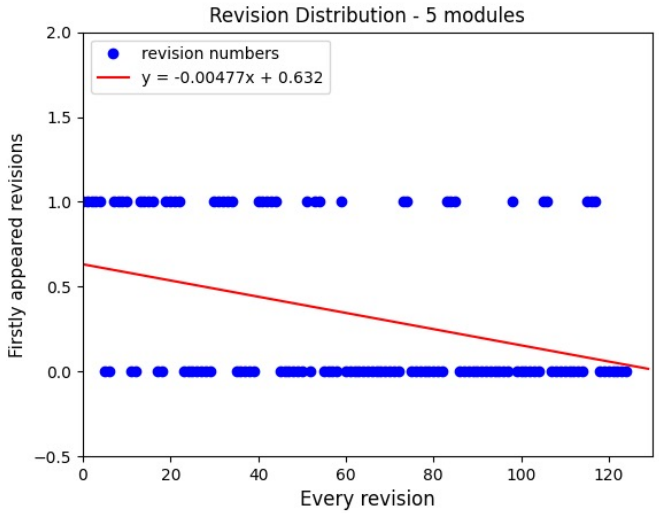
There are some situations that requirement tradeoff may not work properly:

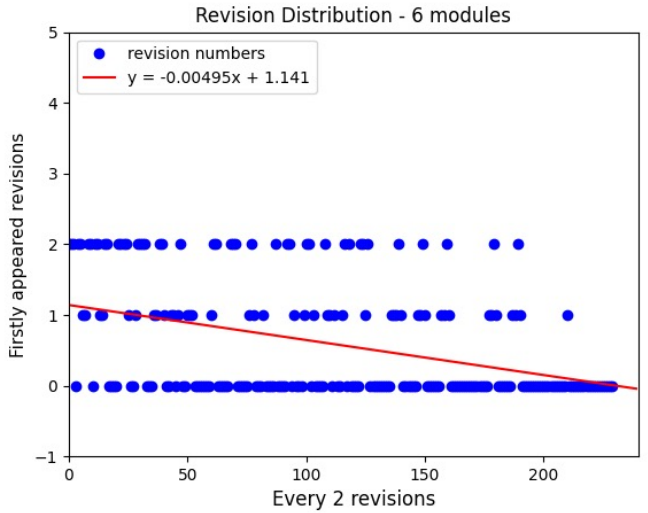
I. The weights of requirements are all the same or ambiguous to declare. In this case, the principle based on the priority loses its meaning.
II. When the requirements are not independent to others. That is, the waive of a lower-weighted requirement may affect the effectiveness of other important ones.

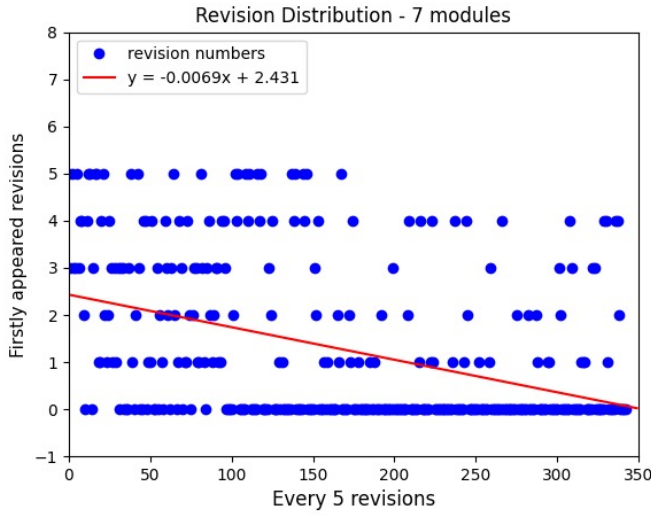
Appendix

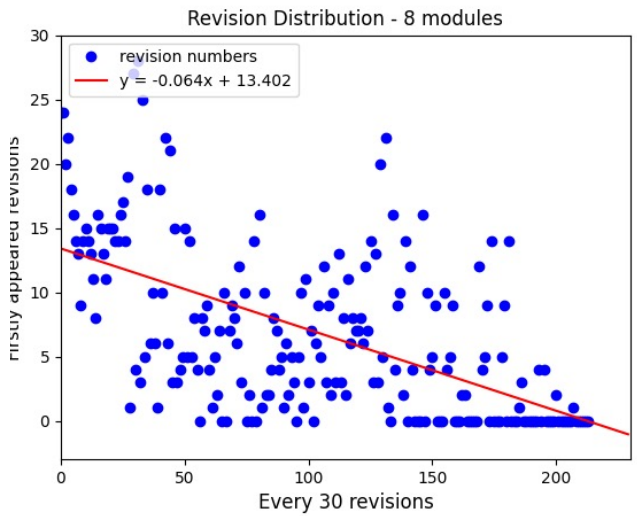
Explanation for evaluation result of OASIS

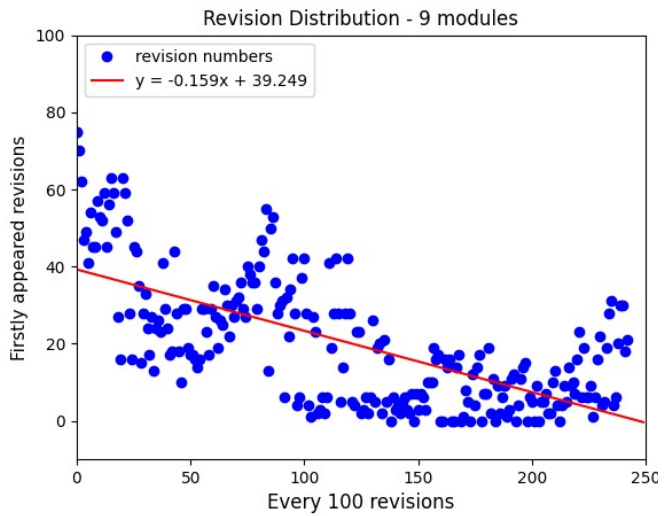












Appendix

Theoretical proof of the performance comparison

$$\frac{t_{avg}^{OASIS}}{t_{avg}^{OACAL}} = \frac{\frac{1}{T}\sum_i^T \left(pos_r(R_i)cov_r(N)\right)t_{DCS}}{\frac{1}{T}\left\{\sum_j pos_{nr}(R_j)cov_{nr}(N)t_{MC} + \sum_k \left\{\tau cov_{nr}(N)t_{MC} + \mathcal{R}CKd\|\mathbf{x_N}\|_0 \log\left(\tau cov_{nr}(N)\right)\right\}\right\}} \tag{6.7}$$

$$\frac{t_{max}^{OASIS}}{t_{max}^{OACAL}} = \frac{\max\limits_{i\in(0,T)} \left(pos_r(R_i)cov_r(N)\right)t_{DCS}}{\tau cov_{nr}(N)t_{MC} + \mathcal{R}CKd\|\mathbf{x_N}\|_0 \log(\tau cov_{nr}(N))} \tag{6.8}$$

Example calculation:

$$\frac{t_{avg}^{OASIS}}{t_{avg}^{OACAL}} = \frac{\frac{1}{T}\sum_i^T \left(pos_r(R_i)cov_r(N)\right)t_{DCS}}{\frac{1}{T}\left\{\sum_j pos_{nr}(R_j)cov_{nr}(N)t_{MC} + \sum_k \left\{\tau cov_{nr}(N)t_{MC} + \mathcal{R}CKd\|\mathbf{x_N}\|_0 \log\left(\tau cov_{nr}(N)\right)\right\}\right\}}$$

$$= \frac{471t_{DCS}}{\frac{1}{1000}\left\{\sum_{J_2}^{\frac{1}{}}\tau cov_{nr}(N)t_{MC} + \sum_k \left\{\tau cov_{nr}(N)t_{MC} + \mathcal{R}CKd\|\mathbf{x_N}\|_0 \log\left(\tau cov_{nr}(N)\right)\right\}\right\}}$$

$$= \frac{471t_{DCS}}{\frac{1}{1000}\left\{\sum_{J_2}^{\frac{1}{}} 0.3\times429t_{MC} + \sum_k \left\{0.3\times429t_{MC} + 6\times1.12\times10^{-5}\times100\times6\times7\times\log(0.3\times429)\times10^3\right\}\right\}}$$

$$= \frac{471t_{DCS}}{\left\{(1-0.15)\times0.3\times429t_{MC} + 0.7\times6\times1.12\times10^{-5}\times100\times6\times7\times\log(0.3\times429)\times10^3\right\}}$$

$$= \frac{471t_{DCS}}{109.4t_{MC} + 1384.53}$$

$$\frac{t_{max}^{OASIS}}{t_{max}^{OACAL}} = \frac{\max\limits_{i\in(0,T)} \left(pos_r(R_i)cov_r(N)\right)t_{DCS}}{\tau cov_{nr}(N)t_{MC} + \mathcal{R}CKd\|\mathbf{x_N}\|_0 \log(\tau cov_{nr}(N))}$$

$$= \frac{1693t_{DCS}}{0.3\times429t_{MC} + 6\times1.12\times10^{-5}\times100\times6\times7\times\log(0.3\times429)\times10^3}$$

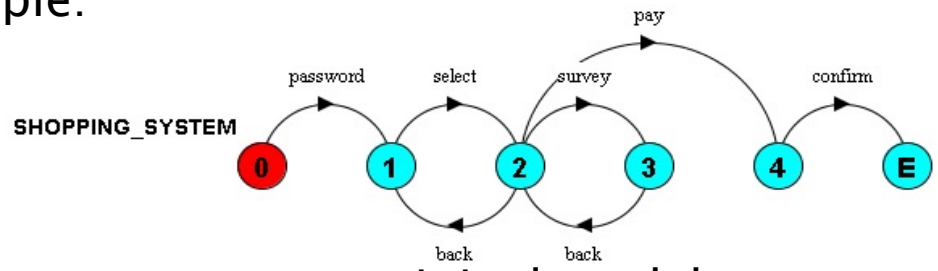
$$= \frac{1693t_{DCS}}{128.7t_{MC} + 1977.90}$$

Appendix

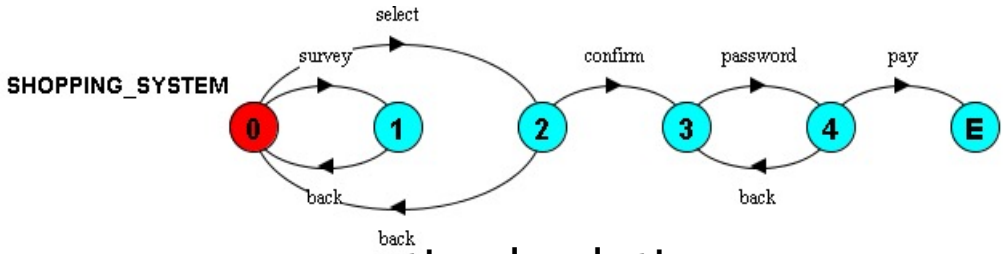
Drawbacks of OACAL

1. In some cases that module consistency is not considered as an important factor, the focus on module-consistent revisions makes OACAL unable to spot some better solutions.

Example:



original model



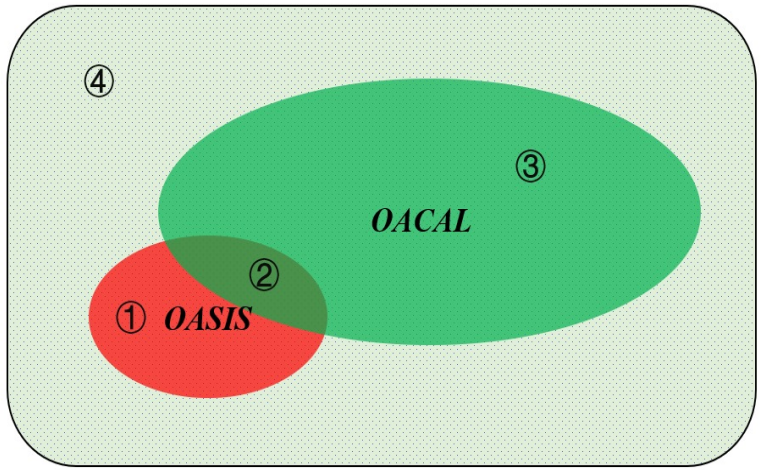
optimal solution

2. Although the evaluation results show a promising accuracy of the trained classification model, when implement machine learning into the practical use without any validation data, the accuracy could not be guaranteed.

Appendix

Which is better? – OASIS or OACAL?

It is arbitrary to say which is better, since they have different focuses.



Coverage of revisions

Area ①
Covers the revisions that in OASIS's coverage but out of OACAL's, which are the ones without module consistency but remain the absolute partial orders.

Area ②
Covers the revisions that remain both module-consistency and partial orders. In the previous evaluation of searching speed, this area was used as the revision set for both approaches.

Area ③
Contains the revisions that are module-consistent but not the partial orders of the actions within them are not absolutely in the order of the original behavior model.

Area ④
Denotes the revisions out of both OASIS's and OACAL's searching scope, which take care of neither module consistency nor partial orders.